

HomeHive

INTRODUCTION:

HomeHive is a unified home-services booking and recommendation platform that connects customers with trusted and verified home-service professionals. Its core promise is to help users find the best available professionals near them quickly, transparently, and reliably.

The platform is designed as a location-aware marketplace. Customers can search for household services, compare professionals, view ratings and reviews, check availability, understand pricing, book a service, track the booking, and submit reviews. Professionals can join the platform, create their profiles, complete verification, receive bookings, and grow their business. Administrators manage users, providers, services, bookings, complaints, ratings, reports, and platform configuration.

The main value of HomeHive is fast discovery, location-aware recommendations, real-time availability, verified professionals, transparent pricing, and flexible scheduling.

SCENARIO Based Case Study:

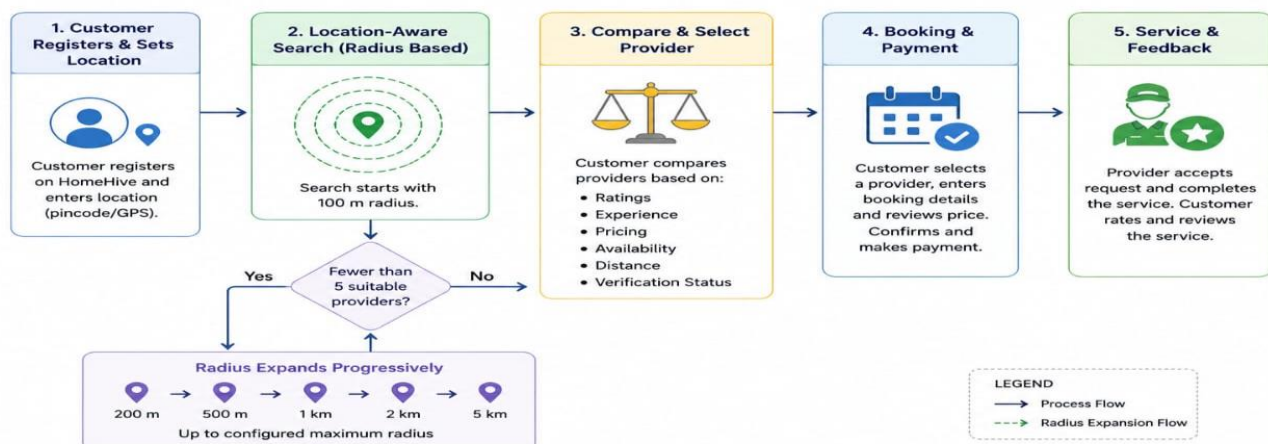
Consider a customer who needs a trusted home-service professional. The customer registers on HomeHive and searches for the required service using a location such as a pincode.

HomeHive obtains the user's location and begins searching for suitable providers near the customer. The search starts with a 100 m radius. If fewer than five suitable providers are found, the radius progressively expands to 200 m, 500 m, 1 km, 2 km, 5 km, and up to the configured maximum radius.

The customer can compare professional profiles, ratings, experience, pricing, availability, verification status, and distance. Search results are available through list and map views. The customer selects a suitable professional, enters booking details such as service, date, start time, duration, number of days, number of people, food requirement, and tip, and reviews the calculated price before confirming.

The provider receives the request and can accept or reject it. The booking progresses through its defined lifecycle. After completion, the customer can provide a rating and review.

HomeHive – Location-Aware Search & Booking Process



SYSTEM REQUIREMENTS:

Software Requirements:

- Frontend: React + Vite.
- Routing: React Router.
- State Management: Redux Toolkit.
- API Communication: Axios.
- Styling: Tailwind CSS.
- Backend Runtime: Node.js.
- Backend Framework: Express.js.
- APIs: REST APIs.
- Authentication: JWT authentication.
- Authorization: Role-based authorization.
- Database: MongoDB with Mongoose.
- Image and File Storage: Cloudinary.
- Maps: OpenStreetMap + Leaflet + React-Leaflet.
- API Testing: Postman can be used during development.
- Source Control: Git/GitHub can be used for project version management.

Hardware Requirements:

- A modern development computer capable of running a browser, Node.js development server, database tools, and an IDE simultaneously.
- Recommended RAM: 8 GB or higher.
- Stable internet connection for development dependencies, external map resources, and cloud image storage.
- Modern browser for testing responsive web interfaces.

PROJECT ARCHITECTURE:

HomeHive uses a full-stack client-server architecture. The client runs in the browser and is developed using React, Vite, React Router, Redux Toolkit, Axios, and Tailwind CSS.

The client communicates with an API gateway/backend built with Node.js and Express.js. The backend provides authentication, provider search and recommendation, availability management, booking management, reviews and ratings, and administration.

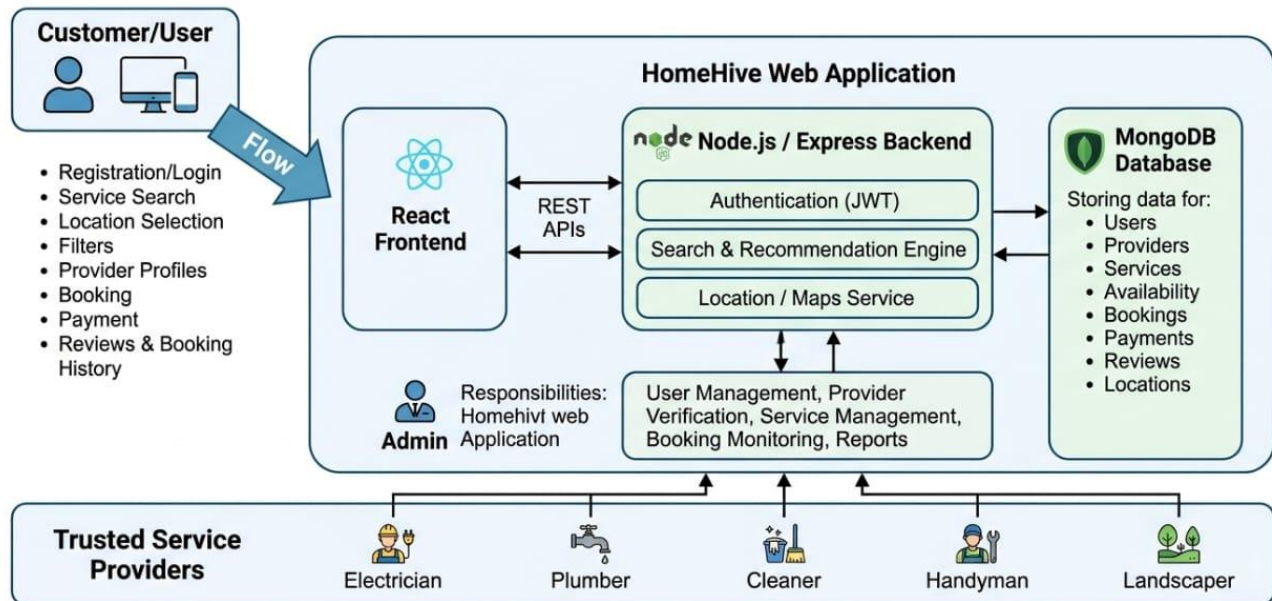
MongoDB with Mongoose provides persistent data storage. Cloudinary is used for profile, verification, service, and portfolio images. OpenStreetMap with Leaflet and React-Leaflet provides map visualization without requiring a paid Google Maps API.

The architecture is designed around role-based access for customers, professionals, and administrators.

HOMEHIVE PLATFORM ARCHITECTURE

Academic Software Project Diagram

HomeHive unified home-services booking and recommendation platform



TECHNICAL ARCHITECTURE:

The technical flow is:

CLIENT (BROWSER)

- React + Vite
- React Router
- Redux Toolkit
- Axios
- Tailwind CSS

API GATEWAY / BACKEND

- Node.js
- Express.js
- REST APIs
- JWT Authentication
- Role-Based Authorization

CORE BACKEND SERVICES

- Authentication
- Provider Search / Recommendation
- Availability Management
- Booking Management
- Reviews & Ratings
- Admin Management

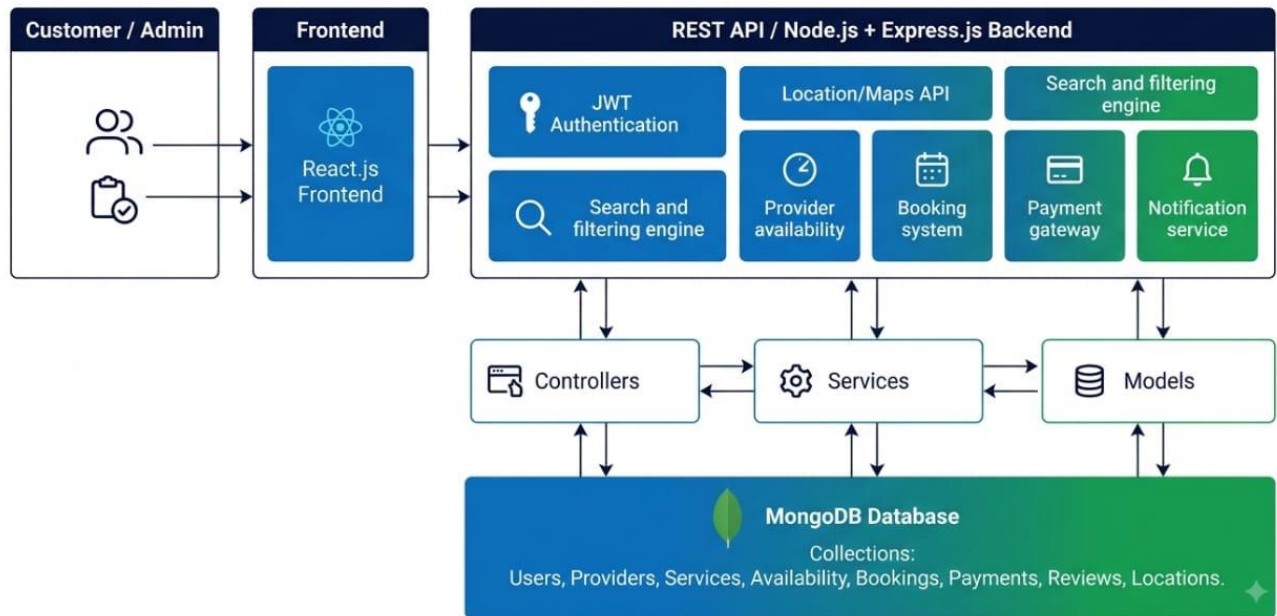
DATABASE / EXTERNAL SERVICES

- MongoDB + Mongoose

- Cloudinary for Images & Files
- OpenStreetMap + Leaflet + React-Leaflet for Maps
- Email/SMS notifications can be integrated as an external service.

The security layer includes password hashing, protected routes, input validation, injection protection, CORS, environment variables, and keeping secrets out of the frontend.

HomeHive | MERN-stack Location-aware home-services booking platform



ER DIAGRAM:

The presentation specifies the main business data required by HomeHive even though a graphical ER diagram is not included in the supplied slides. The document therefore represents the data relationships implied by the project requirements.

Core entities:

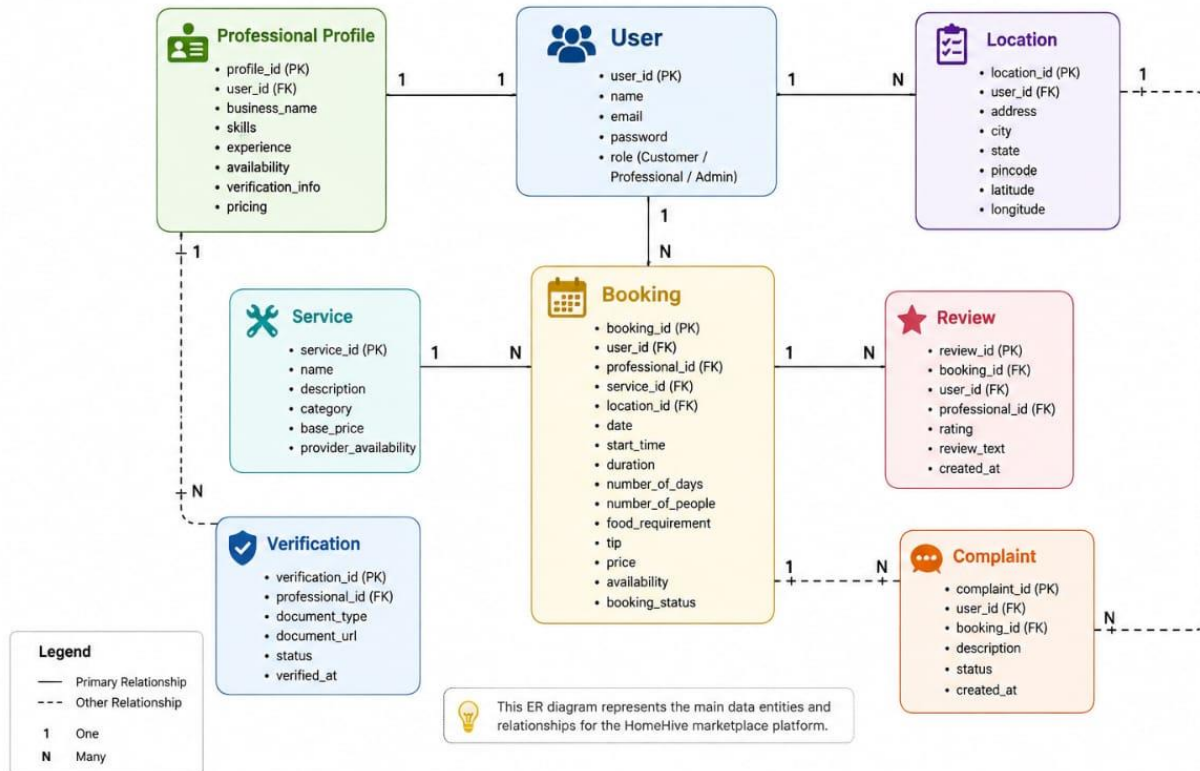
- **User** – represents customers, professionals, and administrators. Role-based login redirects each user to the appropriate dashboard.
- **Professional Profile** – stores business/provider profile information, skills, experience, availability, verification information, and pricing.
- **Service** – represents the household services offered through the marketplace.
- **Location** – represents the customer's or professional's geographic position. Provider search uses MongoDB GeoJSON Point data and a 2dsphere index.
- **Booking** – stores service, date, start time, duration, number of days, number of people, food requirement, tip, price, provider availability, and booking status.
- **Review** – stores ratings and reviews associated with completed services.
- **Verification** – represents provider identity/document verification.
- **Complaint** – represents platform complaints managed by the administrator.

Relationships:

- A User can act as a Customer, Professional, or Admin according to role.
- A Professional has a Professional Profile and can provide services.
- A Customer creates Bookings.
- A Professional receives Bookings.
- A Booking refers to a Service and has a lifecycle.
- Completed Bookings can receive Reviews.
- Professional profiles contain Location information used by geographic search.
- Administrators manage users, providers, services, bookings, complaints, ratings, reports, and configuration.

HomeHive ER Diagram

The diagram below shows the core entities and relationships for the HomeHive platform.



Key Features:

1. Fast Service Discovery

Customers can explore a wide range of household services easily.

2. Location-Aware Search

HomeHive uses the customer's location and searches for nearby professionals.

3. Progressive Radius Expansion

The system starts with a 100 m radius and expands progressively when fewer than five suitable providers are found.

4. Provider Comparison

Customers can compare profiles, ratings, experience, pricing, distance, availability, and verification.

5. Recommendation and Ranking

Providers are ranked using distance, availability, rating, experience, price, profile quality, and verification.

6. Clear Provider Labels

The system can display labels such as Best Match, Nearest, Top Rated, Best Value, and Available Now.

7. Transparent Distance-Based Pricing

The estimated rate is calculated from the base hourly rate plus a configurable distance adjustment.

8. Trust and Verification

Verified and currently available professionals receive priority in the results.

9. Booking and Availability

Customers can enter service, date, start time, duration, number of days, number of people, food requirement, and tip.

10. Booking Lifecycle

Bookings can move through Pending, Confirmed/Provider Accepted, In Progress, Completed, Rejected, Cancelled, and Disputed states.

11. Dynamic Price Calculation

The estimated price updates when booking inputs change.

12. Map Visualization

Customers can see suitable providers on a map using OpenStreetMap and Leaflet.

13. Reviews and Ratings

Customers can provide genuine ratings and reviews after service completion.

14. Role-Based Dashboards

Customers, professionals, and administrators are redirected to their appropriate dashboards.

ROLES AND RESPONSIBILITIES:

Customer:

1. Register – Create an account.
2. Search – Search for required home services.
3. Compare – Compare professionals based on relevant information.
4. View Profile – Check provider details, ratings, experience, pricing, and availability.
5. Book – Enter the required booking information and confirm.
6. Track – Monitor booking and service status.
7. Review – Submit ratings and reviews.

Professional:

1. Join – Enter the platform as a service professional.
2. Sign Up – Create a professional account.
3. Create Profile – Add professional information.
4. Complete Onboarding – Complete required provider information.
5. Get Verified – Complete identity/document verification.
6. Receive Bookings – Receive booking requests.
7. Earn – Provide services and grow the business.

Admin:

1. Manage Users.
2. Manage Providers.
3. Manage Services.
4. Manage Bookings.
5. Manage Complaints.
6. Manage Ratings.
7. Manage Reports and Analytics.
8. Manage Platform Configuration.

Role-based login redirects each user to the appropriate Customer Dashboard, Professional Dashboard, or Admin Dashboard.

User Flow:

Customer Flow:

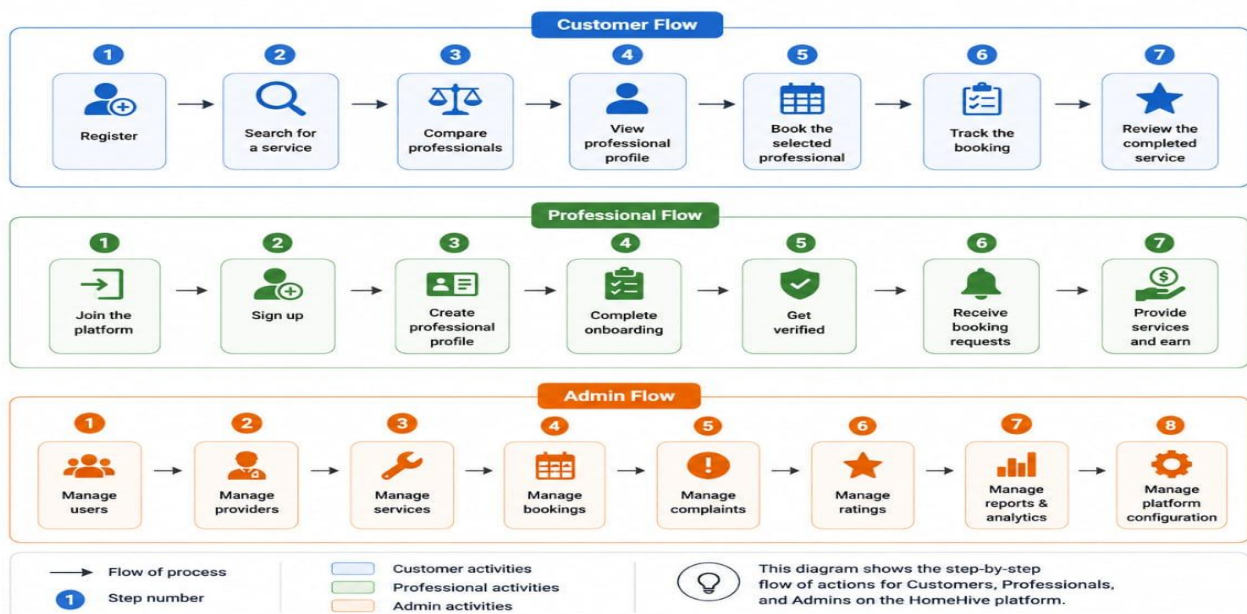
1. Register.
2. Search for a service.
3. Compare professionals.
4. View a professional profile.
5. Book the selected professional.
6. Track the booking.
7. Review the completed service.

Professional Flow:

1. Join the platform.
2. Sign up.
3. Create a professional profile.
4. Complete onboarding.
5. Get verified.
6. Receive booking requests.
7. Provide services and earn.

Admin Flow:

1. Manage users.
2. Manage providers.
3. Manage services.
4. Manage bookings.
5. Manage complaints.
6. Manage ratings.
7. Manage reports and analytics.
8. Manage platform configuration.



MVC PATTERN:

HomeHive can be organized using the MVC pattern to separate data, business logic, and request handling.

Model Layer:

MongoDB and Mongoose models represent application data such as users, professional profiles, services, locations, bookings, reviews, verification information, and complaints.

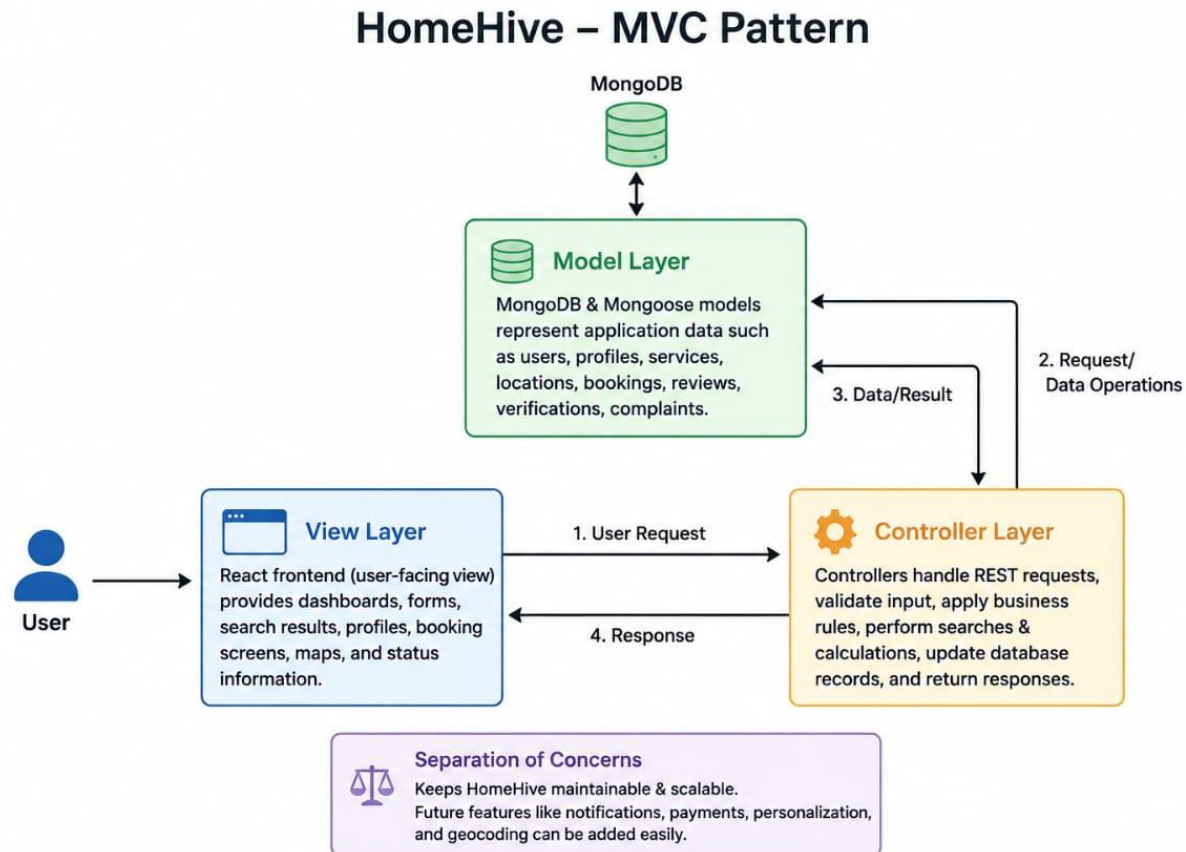
Controller Layer:

Controllers receive requests from REST routes, validate input, apply business rules, perform searches and calculations, update database records, and return responses.

View Layer:

The React frontend acts as the user-facing view. It provides customer, professional, and admin dashboards, forms, search results, profiles, booking screens, maps, and status information.

The separation of concerns helps HomeHive remain maintainable and scalable as features such as notifications, payment gateways, richer personalization, and geocoding are added in the future.



Project Setup And Configuration.

Project Setup and Configuration:

1. Create the HomeHive project directory.
2. Create separate frontend and backend areas.
3. Initialize the React + Vite client.
4. Initialize the Node.js + Express backend.
5. Install required frontend dependencies.
6. Install required backend dependencies.
7. Configure MongoDB connection.
8. Configure Cloudinary credentials through environment variables.
9. Configure JWT authentication.
10. Configure role-based protected routes.
11. Configure OpenStreetMap/Leaflet map integration.
12. Configure environment variables and keep secrets outside frontend source code.
13. Run frontend and backend development servers.
14. Test APIs and application flows before deployment.

Smart Location-Based Search:

Smart Location-Based Search:

Demonstration Location:

Dhanalakshmi Srinivasan College of Engineering, Coimbatore.

Pincode: 641105.

Map Technology:

OpenStreetMap + Leaflet + React-Leaflet.

Search Strategy:

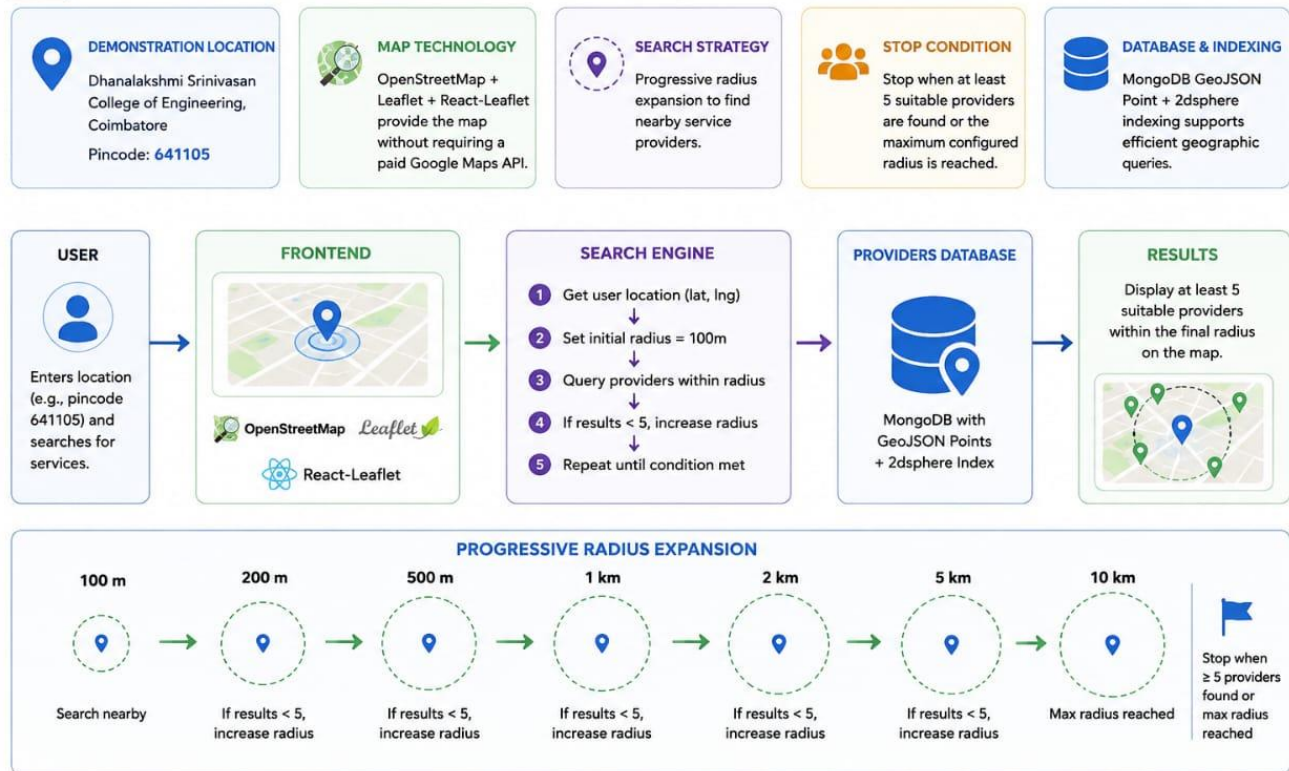
1. Get the user's latitude and longitude.
2. Set the initial search radius to 100 m.
3. Query providers within the current radius.
4. If fewer than five suitable providers are found, increase the radius.
5. Repeat the search.
6. Stop when at least five suitable providers are found or the maximum configured radius is reached.
7. Rank the providers.
8. Display the results on the map and in the result list.

Progressive Radius Expansion:

100 m → 200 m → 500 m → 1 km → 2 km → 5 km → 10 km maximum configured radius.

The presentation specifies that the search should find suitable providers without requiring a paid Google Maps API.

Smart Location-Based Search



Recommendation, Pricing & Trust:

Recommendation, Pricing and Trust:

Provider Ranking:

Ranking considers:

- Distance.
- Availability.
- Rating.
- Experience.
- Price.
- Profile quality.
- Verification.

Example Provider Labels:

- Best Match.
- Nearest.
- Top Rated.
- Best Value.
- Available Now.

Distance-Based Pricing:

The estimated rate is calculated from the provider's base hourly rate plus a distance adjustment.

Example configurable distance multipliers:

- 100 m: 1.00x.
- 200 m: 1.05x.
- 500 m: 1.10x.
- 1 km: 1.20x.
- 2 km: 1.35x.
- 5 km: 1.60x.
- 10 km: 2.00x.

The presentation describes these values as an example configuration. The system should show Base Price + Distance Adjustment + Estimated Rate, with no hidden surcharges.

Priority is given to verified, currently available, relevant, nearby professionals.

Booking & Availability:

Booking and Availability:

Booking captures:

- Service.
- Date.
- Start time.
- Duration.
- Number of days.
- Number of people required.
- Food requirement.
- Tip.

Dynamic Price Calculation:

The estimated price updates automatically when booking inputs change.

Before Confirmation:

- Check provider availability.
- Check existing bookings.
- Check requested date and time.
- Check duration.
- Check other requirements.

Booking Lifecycle:

1. Pending – Booking request created.
2. Confirmed / Provider Accepted – Provider accepts the request.
3. In Progress – Service is currently being performed.
4. Completed – Service is completed successfully.

Other States:

- Rejected.
- Cancelled.
- Disputed.

Real-Time Updates:

Customers receive booking status updates at every stage. Professionals can accept or reject requests.

The screenshot shows a mobile application interface for booking a service. The central panel displays the profile of 'Sathish Ram', a Plumber with a rate of ₹320/hr. The booking form includes fields for 'Service' (Plumber), 'Preferred date' (12-02-2005), and 'Time' (09:00). There are 'Add-ons' for 'Urgent repair' (checked) and 'Monthly maintenance' (unchecked), along with 'Premium support' (unchecked). A 'Contact number' field is present with a placeholder 'Enter your phone number'. Below this is an 'Additional details' section with a text area 'Describe your issue or preferred schedule'. At the bottom are 'Cancel' and 'Confirm booking' buttons. The background shows a blurred view of the app's main screen with a search bar and a list of service providers.

BACKEND DEVELOPMENT:

Backend Development:

The HomeHive backend is built with Node.js and Express.js and exposes REST APIs.

Core Backend Services:

- Authentication – registration, login, JWT authentication, and protected routes.
- Provider Search / Recommendation – geographic search, radius expansion, ranking, and filtering.
- Availability Management – checks whether professionals can accept requested work.
- Booking Management – creates and manages booking lifecycle states.
- Reviews & Ratings Management – records customer feedback.
- Admin Management – manages users, providers, services, bookings, complaints, ratings, reports, and configuration.

The backend also performs price calculation using the provider's base hourly rate and configurable distance multipliers.

Before booking confirmation, the system checks provider availability, existing bookings, requested date and time, duration, and other requirements.

The screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left displays the project structure for 'HomeHive', including folders like 'client', 'server', and 'utils'. The main editor area shows the 'package-lock.json' file for the 'server' directory. The file contains a JSON object with fields like 'name', 'version', 'lockfileVersion', 'requires', and 'packages'. The 'packages' field is an object with a 'homehive-server' entry. The bottom panel shows the 'TERMINAL' tab with a PowerShell prompt. The terminal output shows the execution of 'git push -u origin main', which successfully pushes the code to the 'main' branch of the 'origin' remote.

```
{
  "name": "homehive-server",
  "version": "1.0.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "homehive-server",
      "version": "1.0.0",
      "dependencies": {
        "bcryptjs": "^2.4.3",
        "cloudinary": "^1.38.0",
        "cors": "^2.8.5",
        "dotenv": "^16.3.1",
        "express": "^4.18.4",
        "express-async-errors": "^3.1.1",
        "joi": "^17.10.1",
        "jsonwebtoken": "^9.0.2",

```

```
PS C:\Users\abhis\Downloads\HomeHive> git push -u origin main
Enumerating objects: 85, done.
Counting objects: 100% (85/85), done.
Delta compression using up to 8 threads
Compressing objects: 100% (80/80), done.
Writing objects: 100% (85/85), 17.17 MiB | 5.11 MiB/s, done.
Total 85 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), done.
To https://github.com/ABHISHEK-prince06/HomeHive.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\abhis\Downloads\HomeHive>
```

DATABASE DEVELOPMENT:

Database Development:

HomeHive uses MongoDB with Mongoose.

Geographic Data:

Provider locations are represented using MongoDB GeoJSON Point data. A 2dsphere index supports efficient geographic queries.

The database supports the following business information:

- User accounts and roles.
- Professional profiles.
- Services.
- Geographic locations.
- Availability.
- Bookings and booking status.
- Ratings and reviews.
- Provider verification.
- Complaints.
- Reports and administrative configuration.

The progressive location search depends on geographic indexing so that providers can be queried efficiently at increasing distances.

The supplied presentation specifies MongoDB, GeoJSON Point, and a 2dsphere index but does not provide a final database schema or field-by-field implementation. Those implementation details should be finalized during development.

FRONT-END DEVELOPMENT:

Front-End Development:

The HomeHive frontend uses React + Vite.

Main Frontend Areas:

Customer:

- Registration and Login.
- Search interface.
- Search results.
- Provider comparison.
- Professional profile.
- Booking form.
- Booking tracking.
- Reviews and ratings.
- Customer dashboard.

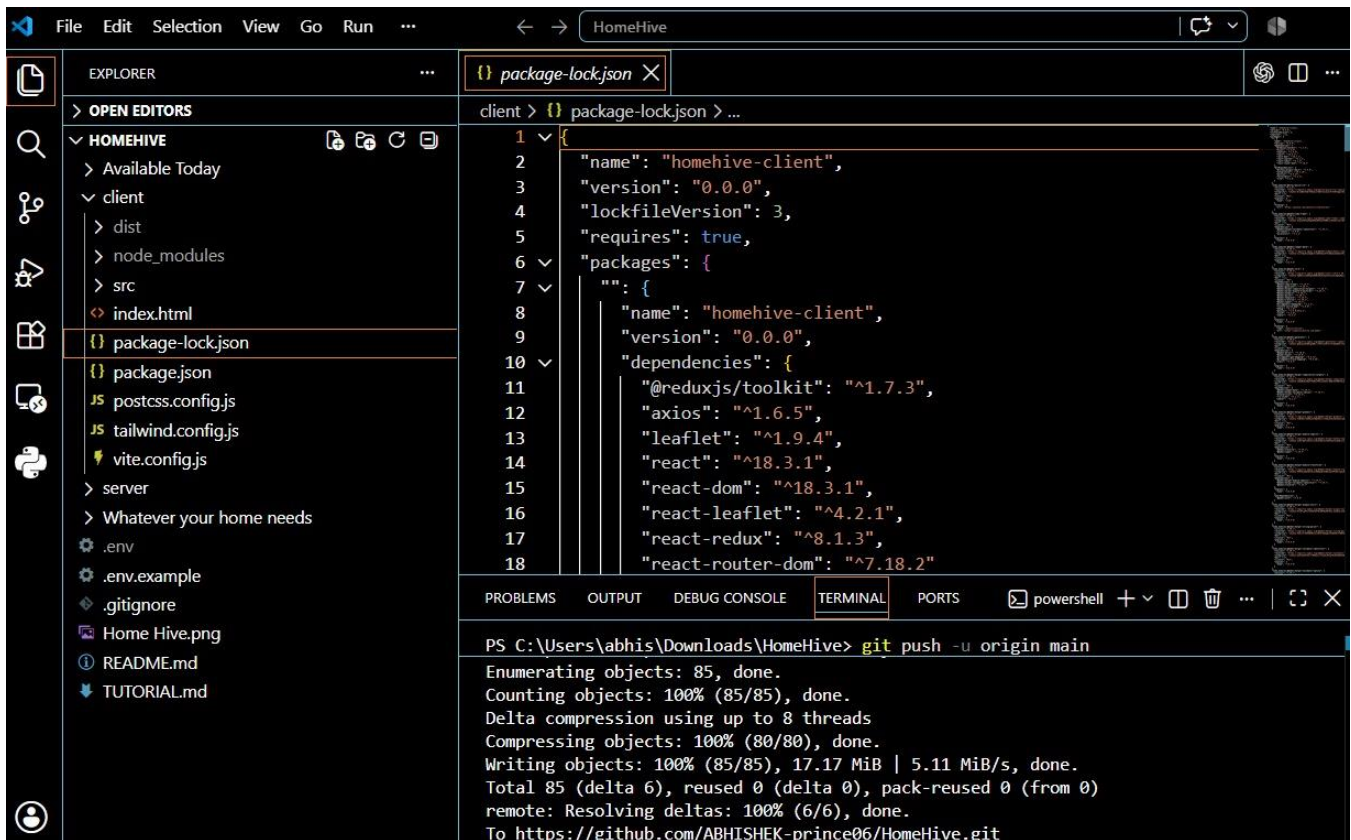
Professional:

- Join as Professional.
- Registration.
- Professional profile.
- Onboarding.
- Verification.
- Booking requests.
- Booking status.
- Earnings/business dashboard.

Admin:

- User management.
- Provider management.
- Service management.
- Booking management.
- Complaint management.
- Rating management.
- Reports and analytics.
- Platform configuration.

React Router handles navigation. Redux Toolkit handles shared application state. Axios communicates with REST APIs. Tailwind CSS provides responsive styling.



SECURITY & BEST PRACTICES:

Security and Best Practices:

The presentation identifies the following security measures:

- Password hashing.
- Protected routes.
- Input validation.
- Injection protection.
- CORS.
- Environment variables.
- No secrets stored in frontend code.
- JWT authentication.
- Role-based authorization.

The security design protects account access and restricts operations according to customer, professional, and administrator roles.

MVP ROADMAP:

MVP Roadmap – Build Order:

1. Setup / Authentication – Project setup and secure authentication.
2. Models / Seed Data – Data models and seed initial provider data.

3. Search – Core search functionality.
4. Maps – Map integration and visualization.
5. Radius Expansion – Expand search by geographic radius.
6. Ranking – Rank results by relevance and quality.
7. Filters – Advanced filters such as price and rating.
8. Profiles – Business/provider profiles.
9. Booking – Booking and availability.
10. Dashboards – Admin and user dashboards.
11. Reviews – Ratings and reviews.
12. Security – Security, roles, and permissions.
13. Push Notifications.
14. Testing – QA, bug fixes, and testing.

Post-MVP / Future:

- Payments.
- Real geocoding.
- Richer personalization.
- Smarter recommendations.
- Notifications.
- Payment gateway integration.

The quality goal is a functioning startup-style full-stack MVP rather than a collection of static screens.

DIFFERENTIATOR:

HomeHive's key differentiator is its progressive geographic search combined with provider availability, distance, price, rating, and relevance.

Instead of simply showing a fixed-radius list, the platform expands its search when enough suitable professionals are not available. This approach is intended to produce useful results while keeping the search local and relevant.

The recommendation system gives higher priority to providers who are verified, available, relevant, nearby, and suitable for the customer's requirements.

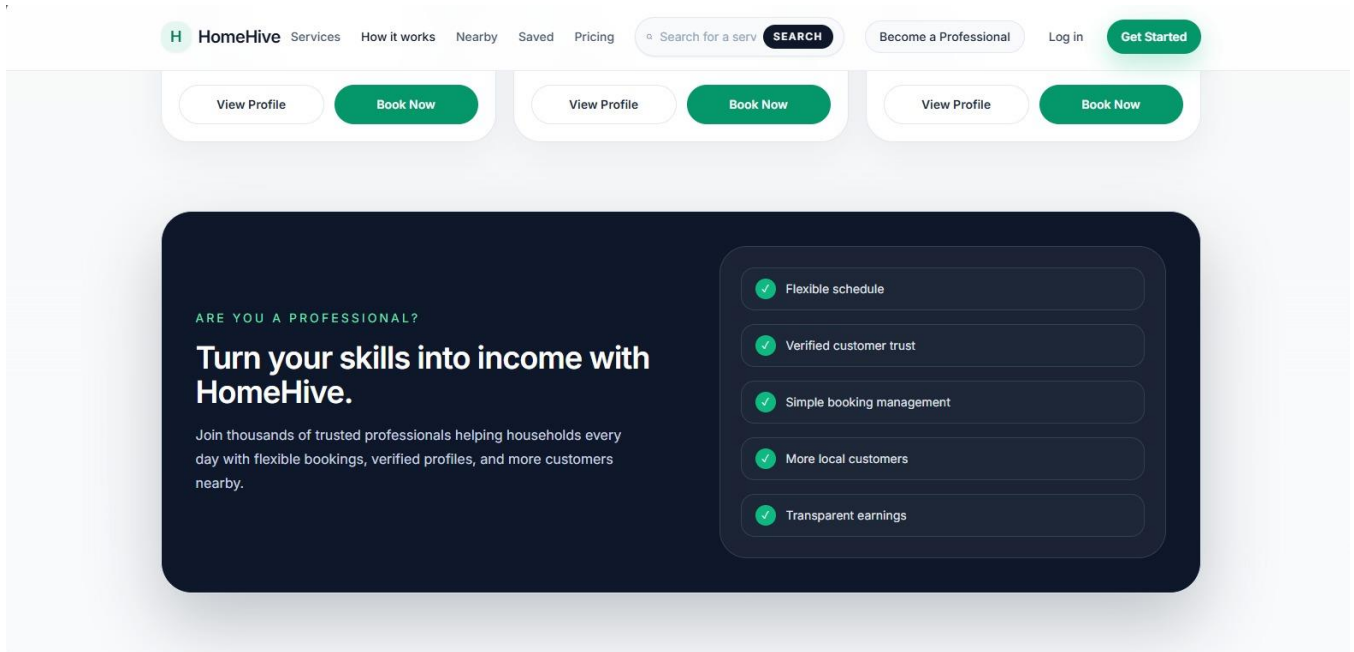
The platform is designed to scale toward advanced capabilities and richer user experiences after the MVP.

Output ScreenShots:

HomeHive Landing / Home Page

[HOMEHIVE APPLICATION SCREENSHOT — INSERT ACTUAL SCREENSHOT HERE]

The supplied 8-slide presentation contains project content and diagrams, not actual application UI screenshots.

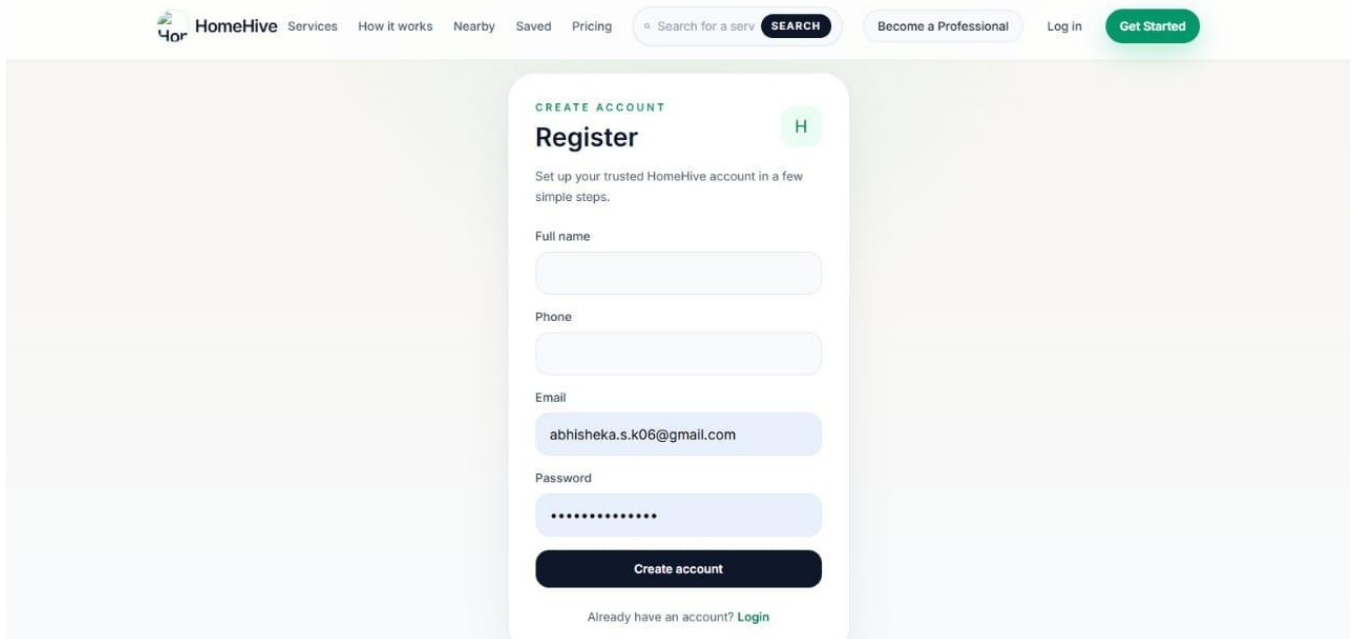


Login.jsx:

Customer Login / Professional Login

[HOMEHIVE APPLICATION SCREENSHOT — INSERT ACTUAL SCREENSHOT HERE]

The supplied 8-slide presentation contains project content and diagrams, not actual application UI screenshots.

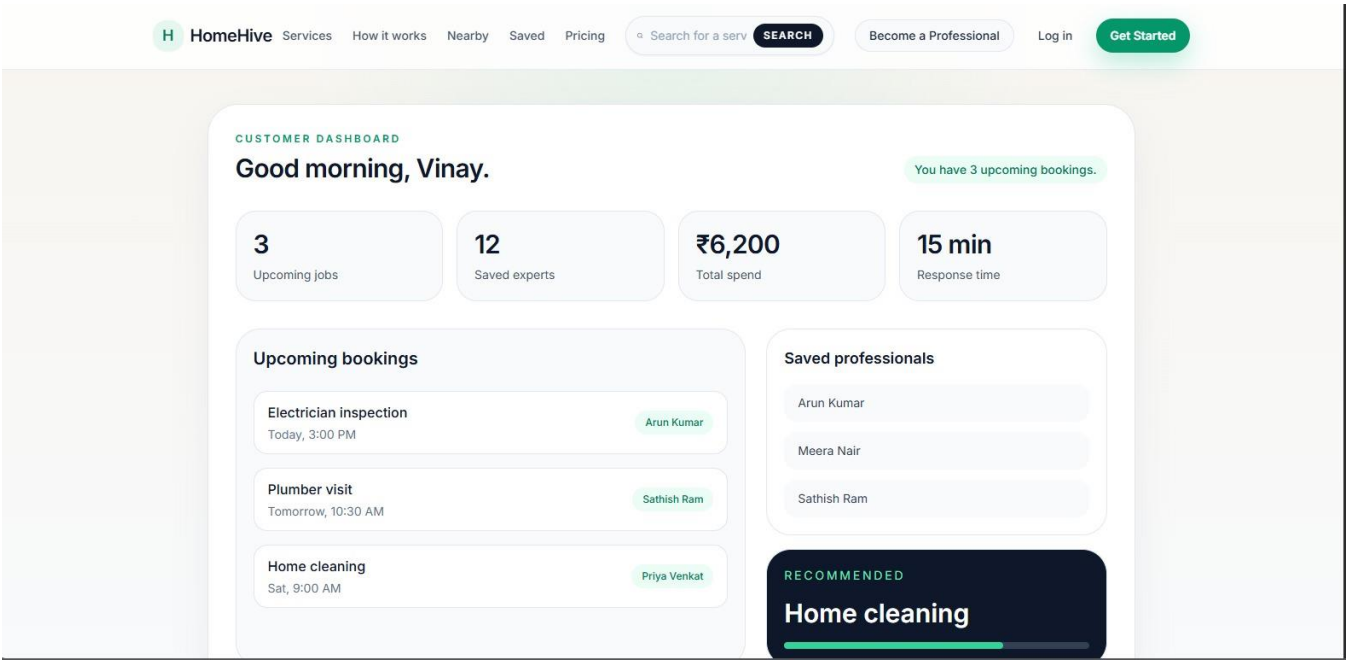


Customer’s Page:

Customer Dashboard / Search Results

[HOMEHIVE APPLICATION SCREENSHOT — INSERT ACTUAL SCREENSHOT HERE]

The supplied 8-slide presentation contains project content and diagrams, not actual application UI screenshots.

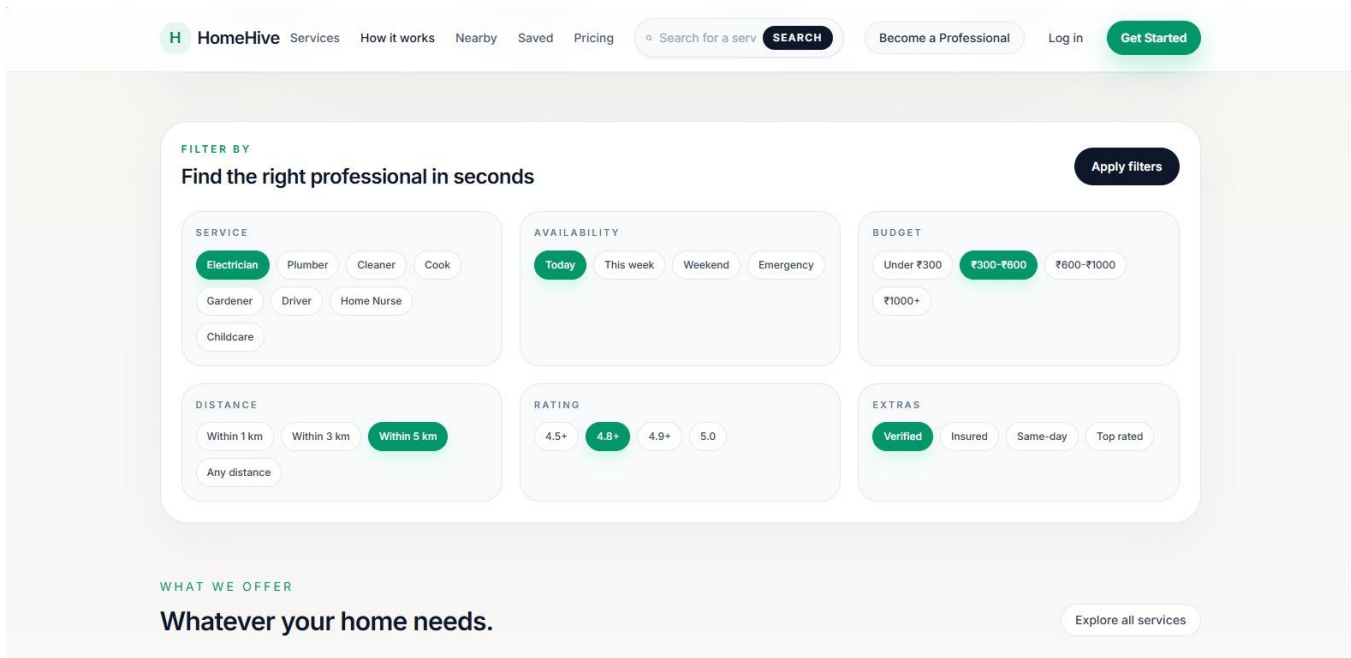


Professional’s Page:

Professional Dashboard / Profile

[HOMEHIVE APPLICATION SCREENSHOT — INSERT ACTUAL SCREENSHOT HERE]

The supplied 8-slide presentation contains project content and diagrams, not actual application UI screenshots.

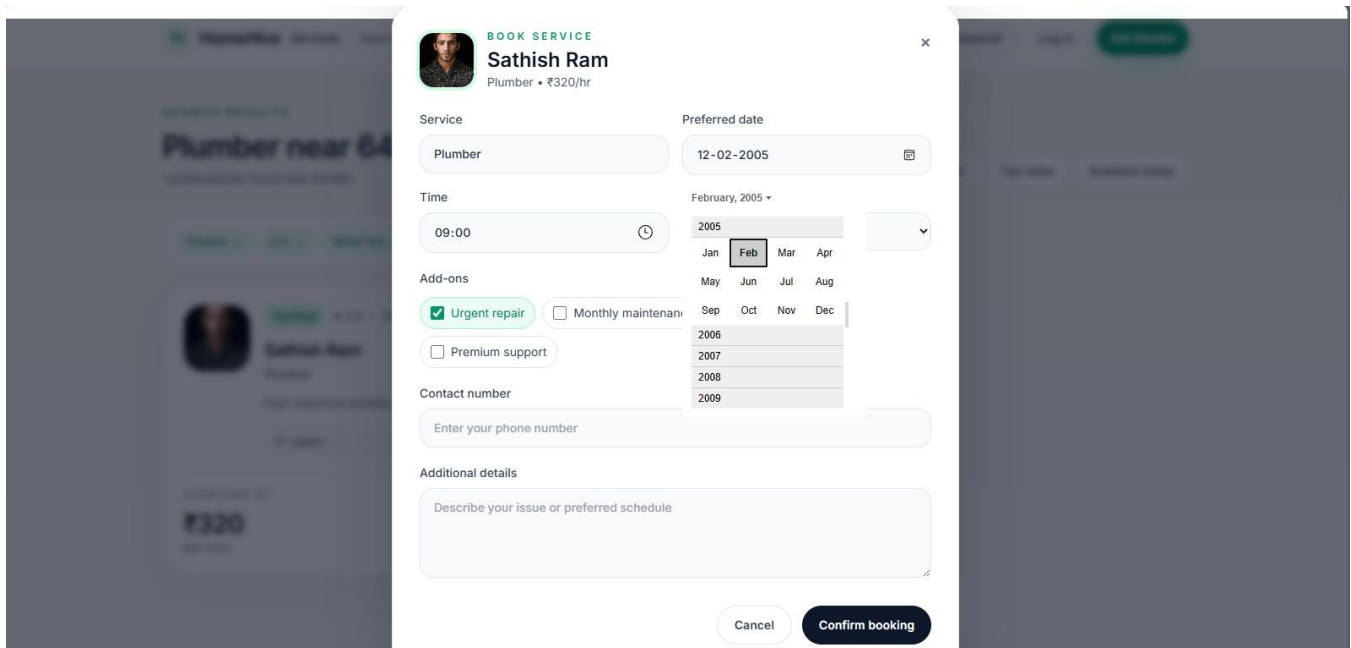


Booking Page:

Booking Form / Booking Summary / Tracking

[HOMEHIVE APPLICATION SCREENSHOT — INSERT ACTUAL SCREENSHOT HERE]

The supplied 8-slide presentation contains project content and diagrams, not actual application UI screenshots.



Admin's Page:

[Admin Dashboard](#) / [Users](#) / [Providers](#) / [Services](#) / [Reports](#)

[HOMEHIVE APPLICATION SCREENSHOT — INSERT ACTUAL SCREENSHOT HERE]

The supplied 8-slide presentation contains project content and diagrams, not actual application UI screenshots.

Demo / Final Information:

Demonstration Location:

Dhanalakshmi Srinivasan College of Engineering, Coimbatore.

Pincode: 641105.

Map Technology:

OpenStreetMap + Leaflet + React-Leaflet.

Demonstration Search:

The customer enters a location and searches for a home service. The search starts at 100 m and progressively expands to 200 m, 500 m, 1 km, 2 km, 5 km, and the maximum configured radius until at least five suitable providers are found or the maximum radius is reached.

Demo Video Link:

<https://drive.google.com/drive/folders/1wgBqRra6NdOmhtmnsavBYcfkyTQWENYH?usp=sharing>

Code Drive / Repository Link:

<https://github.com/ABHISHEK-prince06/HomeHive.git>

Quality Goal:

A functioning startup-style full-stack MVP, not a collection of static screens.